

AREHI-SECOPS

ASEAN Regional E-Commerce Hybrid Infrastructure & Hardened SecOps System

Phase 5 — Automation & SecOps

Full-Depth Automation & Certification Study Guide

Architect / Author: Danial Nur Irfan bin Azeze

Bachelor of Computer Science — Networks & Security, UTM

CompTIA Security+ Certified

Exam objectives referenced: Cisco CCNA 200-301, CompTIA Security+ SY0-701

Tooling: Netmiko 4.5.0, Ansible (cisco.ios collection), Wazuh SIEM (Docker) — written and reviewed, executed live only where genuinely possible

Devices in scope: all 6 (config-parsing/reference only — see Section 3)

Real, live-executed piece: · Ansible installed & syntax-checked for real (WSL) · Wazuh live deployment attempted, honestly documented

Contents

1. Phase 5 Overview
2. A Note on Exam Objective Tags
3. A Note on Scope — the Packet Tracer Real-NIC Bridge Investigation
4. Objective 1 — Netmiko: Config Backup Automation
5. Objective 2 — Netmiko: Bulk ACL Update
6. Objective 3 — Device Inventory Generator (the one that runs)
7. Objective 4 — Ansible: Declarative VLAN & ACL Deployment
8. Objective 5 — Wazuh SIEM Pipeline
9. Design Decisions & Honesty Notes
10. Appendix — Quick Reference

1. Phase 5 Overview

Phase 5 closes the project with the automation and security-operations layer sitting on top of everything built in Phases 1–4: instead of typing the same command 6 times by hand, or manually re-reading every device's addressing plan to build documentation, this phase automates the repetitive parts and centralizes log visibility across the whole topology. It covers two distinct automation paradigms side by side — Netmiko's imperative, one-command-at-a-time CLI scripting and Ansible's declarative, desired-state configuration management — plus a SIEM pipeline demonstrating log correlation, the practical payoff of centralizing everything Phases 1–3 could only ever show one device at a time.

Files in scope

File	Tooling	Live-executed?
<code>python-scripts/backup_configs.py</code>	Netmiko	No — see Section 3
<code>python-scripts/bulk_acl_update.py</code>	Netmiko	No — see Section 3
<code>python-scripts/device_inventory.py</code>	Plain Python (regex/file parsing)	Yes — no live connection needed
<code>ansible-playbooks/deploy_vlan_acl.yml</code>	Ansible (<code>cisco.ios</code>)	Syntax-checked for real via WSL — never applied to a device
<code>logging-dashboard/docker-compose.yml</code>	Wazuh (Docker)	Live deployment attempted, blocked by environment quirks — see Objective 5

2. A Note on Exam Objective Tags

Phase 5 maps almost entirely to two domains: **CCNA 200-301 Domain 6.0 (Automation and Programmability)** — the whole premise of Netmiko vs. Ansible, and the conceptual distinction between imperative CLI scripting and declarative desired-state management — and **Security+ SY0-701 Domain 4.0 (Security Operations)** — the SIEM pipeline, log correlation, and alerting. Unlike Phase 4, AWS SAA-C03 genuinely does not apply anywhere in this phase and is not tagged anywhere below — consistent with this project's stated policy of never forcing a certification tag where it doesn't actually fit.

3. A Note on Scope — the Packet Tracer Real-NIC Bridge Investigation

Decision: the Netmiko and Ansible code in this phase is written to be genuinely correct, real-device-ready automation — but is never executed against a live target.

CONFIRMED VIA DIRECT INVESTIGATION, NOT ASSUMED

Both Netmiko and Ansible's `network_cli` connection plugin require a real, reachable SSH endpoint. Before writing a single line of automation code, this project checked whether Packet Tracer 9.0.0 can actually expose one: the `PT-Cloud` device (WAN Emulation category) was placed on the canvas and its Connections tab inspected directly. Its only connection types are **Frame Relay, DSL, and Cable** — all Packet-Tracer-internal WAN emulation types. There is no option anywhere to bridge to a real host network adapter, unlike older Packet Tracer versions' Cloud-PT feature. This is a confirmed platform capability gap, not a misconfiguration — the same rigorous "verify via direct query, don't assume" discipline applied to the SNMPv3 (`snmp-server ?`) and spanning-tree (`spanning-tree ?`) findings in earlier phases.

Unlike Phase 4, where LocalStack provided a genuine (if partial) live-apply target for the Terraform, there is no equivalent tool available here — no free, local emulator exposes a Cisco-IOS-speaking SSH endpoint the way LocalStack exposes AWS APIs. The honest choice, consistent with every other Packet-Tracer-imposed limitation in this project, is to write correct code and say so plainly rather than pretend it was tested.

THE ONE EXCEPTION, AND WHY IT MATTERS

`device_inventory.py` needs no live connection at all — it parses this repository's own `.cfg` reference files directly. This script **does** genuinely run, and its output (`device_inventory.json`) is committed as real proof, not a hypothetical. Writing and actually running this script surfaced a real parsing bug worth knowing about even outside this project: a naive regex trying to associate each `interface` block with its `ip address` line leaked across block boundaries whenever an interface had no IP of its own, silently attributing a completely different interface's address to it. See Objective 3 for the full story.

4. Objective 1 — Netmiko: Config Backup Automation

`python-scripts/backup_configs.py` — all 6 devices

CCNA 200-301: Domain 6.0 (Automation and Programmability)

Purpose: the automated equivalent of the manual `copy running-config tftp:` objective from Phase 3 — instead of typing that command 6 separate times (once per device, each time prompted interactively for a hostname and filename), one script connects to every device in turn and saves a timestamped backup automatically.

Script structure, word by word

```
with ConnectHandler(**connection_params(device)) as conn:
    conn.enable()
    running_config = conn.send_command("show running-config")

timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H%M%S")
filename = BACKUP_DIR / f"{device['name'].lower()}-{timestamp}.cfg"
filename.write_text(running_config)
```

Term / call	Meaning
<code>ConnectHandler</code>	Netmiko's core class — opens and manages an SSH session to a network device, abstracting away the vendor-specific quirks of Cisco IOS's CLI (paging prompts, enable-mode handling, command echo) behind one consistent Python interface regardless of platform.
<code>with ... as conn:</code>	A context manager — guarantees the SSH session is cleanly closed even if an error occurs partway through, the same principle as always closing a file handle properly rather than leaking an open connection.
<code>conn.enable()</code>	Explicitly enters privileged EXEC mode before running any command that needs it — <code>show running-config</code> requires this, unlike many <code>show</code> commands that work from user EXEC mode.
<code>conn.send_command()</code>	Sends a single command and waits for the device's prompt to return, handling pagination (<code>--More--</code>) automatically so the full output comes back in one call rather than needing manual "press space to continue" handling.
Timestamped filename	Each backup gets its own uniquely-named file rather than overwriting the previous one — makes it possible to compare a device's configuration at two different points in time, directly useful for a security investigation ("what changed, and when").

EXAM ANGLE — WHY THIS MATTERS BEYOND CONVENIENCE

Automated, scheduled config backups are a real Security+ Domain 4.0 concept (configuration/change management, part of security operations) as much as a CCNA automation concept — having a reliable, timestamped history of every device's configuration is what makes "what changed before this incident started" an answerable question during an investigation, rather than a guess.

EXPECTED RESULT (IF EVER RUN AGAINST A REACHABLE DEVICE)

A `backups/` folder containing one timestamped `.cfg` file per device, each one a faithful text copy of that device's `show running-config` output at the moment the script ran.

5. Objective 2 — Netmiko: Bulk ACL Update

`python-scripts/bulk_acl_update.py` — MY-KL-HQ-CORE, PH-MNL-ROAS, TH-BKK-ROAS

CCNA 200-301: Domain 6.0 (Automation and Programmability)

Security+ SY0-701: Domain 3.0 (Security Architecture — network segmentation maintenance)

Purpose: demonstrates why bulk automation matters most exactly where mistakes are costliest — pushing an identical ACL change to multiple devices by hand risks one device getting a typo, or being forgotten entirely. This script pushes the same new deny entry to all 3 devices carrying the GUEST-CONTAINMENT extended ACL in one run.

Commands pushed, word by word

```
ip access-list resequence GUEST-CONTAINMENT 10 10
ip access-list extended GUEST-CONTAINMENT
  40 deny ip 10.10.30.0 0.0.0.255 10.10.50.0 0.0.0.255
```

Term	Meaning
<code>ip access-list resequence</code>	Renumbers every existing entry in steps of 10 (10, 20, 30...) before inserting anything new — run first specifically so the script can predict exactly which sequence number to insert the new line at, regardless of how many entries a given device's ACL happened to accumulate previously. Without this, inserting "before the trailing permit any any" would require first parsing <code>show access-lists</code> output to discover the right number.
Inserting at sequence 40	IOS extended ACLs evaluate entries in sequence-number order, first match wins. Inserting the new deny at 40 places it before the original <code>permit ip any any</code> (now resequenced to 50) — inserting it after would mean it never gets evaluated at all, since the permit-any-any would already have matched and stopped evaluation first.
<code>conn.send_config_set()</code>	Netmiko's method for sending a list of configuration-mode commands as one logical unit, automatically entering and exiting global configuration mode around them.
<code>conn.save_config()</code>	The scripted equivalent of <code>copy running-config startup-config</code> — without this, the change would be lost on the next reload, exactly like any manually-typed config change.

A DEVICE-SPECIFIC DETAIL THIS SCRIPT HAS TO ACCOUNT FOR

MY-KL-HQ-CORE applies GUEST-CONTAINMENT differently than the other two devices — as a Port ACL on GigabitEthernet1/0/10 (the Phase 3 SVI-bug workaround), not on the vlan30 SVI that PH-MNL-ROAS / TH-BKK-ROAS use on their sub-interfaces. This script is still correct for all 3 unchanged, because it only edits the ACL's own entry list — *where* the ACL is applied is a separate configuration concern this script doesn't touch. A script (or Ansible playbook) that also needed to manage ACL *application*, not just its entries, would need to know about this per-device difference explicitly rather than assuming identical devices behave identically.

EXPECTED RESULT (IF EVER RUN AGAINST A REACHABLE DEVICE)

```
show access-lists GUEST-CONTAINMENT
```

Would show the new deny entry at sequence 40 on all 3 devices, with the original 3 denies renumbered to 10/20/30 and the permit-any-any pushed to 50 — identical on all 3, confirming the bulk push was consistent.

6. Objective 3 — Device Inventory Generator (the one that runs)

`python-scripts/device_inventory.py` — parses all 6 `.cfg` files, no live connection

CCNA 200-301: Domain 6.0 (Automation and Programmability — data formats/parsing)

Purpose: generates a structured JSON inventory (hostname, VLANs, routed interfaces with their IPs) by parsing this repository's own `.cfg` reference files directly — genuinely useful even without live devices, since documentation-from-source-of-truth is exactly the kind of task that shouldn't need a live SSH session at all.

A real bug this script's own author caught by actually running it

The first version of this parser split each file into stanzas on lines that were exactly `!`, assuming each interface's header line would be the first line of its own stanza. Running it against `SG-EDGE-GW.cfg` immediately proved that assumption wrong: multi-line `! ----` comment banners commonly appear directly before an `interface` line *without* a blank `!` separator in between, so the interface header ended up buried in the middle of a stanza instead of at the start of one — and the parser's `block[0]` check silently found nothing, dropping every routed interface on that device from the output (3 interfaces became 0). An earlier, even more naive version had the opposite failure mode: a lazy regex spanning multiple lines leaked across interface boundaries entirely, at one point attributing `GigabitEthernet1/0/22`'s real IP address to `Port-channel1` — a Layer 2 interface with no IP address at all.

The fix, word by word

```
for line in text.splitlines():
    if INTERFACE_HEADER_RE.match(line):
        blocks.append([line])
    elif blocks and line.startswith(" "):
        blocks[-1].append(line)
    # any other line ends whatever interface block was open
```

Term	Meaning
Trigger directly off <code>interface</code> lines	Rather than relying on a separator character that turned out not to reliably precede every interface declaration, the corrected version starts a new block the instant it sees an actual <code>interface</code> line — removing the false assumption entirely instead of patching around it.
<code>line.startswith(" ")</code>	Cisco IOS indents every sub-command of an interface stanza with exactly one leading space. Any line that <i>isn't</i> indented (a comment, a blank line, the next top-level command) naturally ends the current interface's block just by not matching this condition — no explicit "end of block" marker needed.

EXAM ANGLE — WHY THIS BELONGS IN A SECURITY-FOCUSED PORTFOLIO, NOT JUST A CODING ONE

An inventory or backup script that silently drops or misattributes data is arguably worse than no automation at all — it creates false confidence that a device's configuration is fully documented when it isn't. Verifying automation output against ground truth (here: manually re-checking the real `.cfg` files line by line after every fix) is the same discipline Security+ Domain 4.0 expects of any automated security tooling, not just this one script.

ACTUAL RESULT (THIS ONE REALLY HAPPENED)

```
Parsed 6 devices -> device_inventory.json
MY-KL-HQ-CORE: 5 VLANs, 8 routed interfaces
PH-MNL-ACC: 5 VLANs, 1 routed interfaces
TH-BKK-ACC: 5 VLANs, 1 routed interfaces
PH-MNL-ROAS: 0 VLANs, 6 routed interfaces
SG-EDGE-GW: 0 VLANs, 3 routed interfaces
TH-BKK-ROAS: 0 VLANs, 6 routed interfaces
```

Every IP address in the final `device_inventory.json` was manually cross-checked against the source `.cfg` files after the fix, confirming an exact match with no leaked or missing entries.

7. Objective 4 — Ansible: Declarative VLAN & ACL Deployment

`ansible-playbooks/deploy_vlan_acl.yml` — all 6 (VLANs), 3 devices (ACL)

CCNA 200-301: Domain 6.0 (Automation and Programmability — declarative vs. imperative)

Purpose: the declarative counterpart to Objectives 1–2's imperative Netmiko scripts — instead of listing the exact commands to run, Ansible's `cisco.ios` collection modules describe the *desired end state*, and Ansible itself calculates and applies only the diff needed to reach it.

Playbook structure, word by word

```
- name: Merge VLAN database
  cisco.ios.ios_vlans:
    state: merged
    config:
      - vlan_id: 10
        name: MGMT
        state: active
      ...

- name: Merge ACL entries
  cisco.ios.ios_acls:
    state: merged
    config:
      - afi: ipv4
        acls:
          - name: GUEST-CONTAINMENT
            acl_type: extended
            aces: [...]
```

Term	Meaning
Imperative (Netmiko) vs. declarative (Ansible)	A Netmiko script says exactly <i>how</i> : which literal CLI commands to send, in order. An Ansible module says exactly <i>what</i> : the end state desired. Ansible itself is responsible for figuring out which underlying commands actually need to run to reach that state — a real device already matching the desired config gets left alone entirely.
<code>state: merged</code>	Adds/updates only what's listed here without removing anything not mentioned — the Ansible equivalent of the "additive, not destructive by default" principle already applied to Terraform's own resource model in Phase 4 (a new <code>aws_security_group_rule</code> doesn't delete unrelated rules either).
<code>ios_vlans</code> vs. <code>ios_acls</code> as separate modules	Each Cisco IOS resource type (VLANs, ACLs, interfaces, ACL-to-interface bindings) gets its own dedicated, purpose-built module in the <code>cisco.ios</code> collection, rather than one generic "run this raw CLI" module for everything — this is what lets Ansible reason about desired state accurately, since a raw CLI wrapper has no structured understanding of what "VLAN 10" or "this ACE" currently is.
<code>hosts:</code> <code>guest_containment_devices</code>	Targets the Ansible inventory group matching exactly the 3 devices with this ACL in Phase 3's actual design (excluding <code>sg-edge-gw</code> , which never had a local GUEST_WIFI VLAN to contain) — the same precise scoping already applied in <code>devices.py</code> 's <code>GUEST_CONTAINMENT_DEVICES</code> list for the Netmiko script.

EXAM ANGLE — IDEMPOTENCY

Re-running this exact playbook against a device that already has the desired VLANs and ACL entries would report `changed: false` for every task — nothing would actually be re-sent to the device.

Idempotency (an operation that produces the same end state no matter how many times it's run) is a core declarative-automation concept tested on CCNA Domain 6.0, and is the single biggest practical difference from a Netmiko script, which would happily re-send the same commands every time regardless of current state.

ACTUAL RESULT (THIS ONE REALLY HAPPENED TOO)

Ansible's control node doesn't officially support native Windows, so it was installed inside WSL Ubuntu instead (`sudo apt install ansible`, which bundled the `cisco.ios` collection automatically).

`ansible-playbook --syntax-check` passed cleanly, and `ansible-inventory -i inventory.ini --list` confirmed all 6 devices resolve with the correct IPs, `ansible_network_os`, and credential-lookup expressions — genuine mechanical verification that both files are correct, even though nothing was ever applied to a live device.

8. Objective 5 — Wazuh SIEM Pipeline

`logging-dashboard/` — Docker Compose, sample logs from all 6 devices

Security+ SY0-701: Domain 4.0 (Security Operations — log aggregation, correlation, alerting)

Purpose: centralizes visibility across every device's security-relevant events in one place — the practical payoff of everything logged individually throughout Phases 1–3 (ACL hits, SSH logins, port security violations) becoming genuinely useful only once it's correlated together rather than scattered across 6 separate device consoles.

Why Wazuh, and why sample logs

Decision	Reasoning
Wazuh over a generic ELK stack	Purpose-built open-source SIEM/XDR with pre-built security correlation rules (brute-force detection, compliance dashboards) mapping directly onto Security+ Domain 4.0 content, rather than a general-purpose log search tool an operator would need to build security-specific rules for from scratch.
Representative sample logs, not a live feed	Real-time syslog forwarding from Packet Tracer would need the same real-NIC bridge confirmed absent in Section 3 - the same limitation blocking live Netmiko/Ansible execution. Sample logs are reconstructed in genuine Cisco IOS syslog format, using this project's own real device names and addressing, clearly labeled as samples rather than a live capture.

A LIVE DEPLOYMENT WAS ACTUALLY ATTEMPTED - AND HONESTLY, IT DIDN'T SUCCEED

Unlike Netmiko/Ansible (never attempted live at all, per Section 3), a genuine attempt was made to stand up this Wazuh stack end-to-end. Three separate Docker Desktop/WSL2 environment quirks were hit and ruled out in turn: (1) initial cert mount paths were guessed wrong, fixed by reading each container's own real config directly; (2) a Java SecurityManager `FilePermission` denial reading a plainly world-readable file, specific to Windows-path bind mounts, that persisted even after switching to Wazuh's own official whole-directory mount pattern; (3) after moving the setup into WSL Ubuntu's native filesystem to rule out the Windows-path hypothesis, a stranger bug appeared - bind-mounted subdirectories were visible inside the container but top-level files sitting in the same directory were not, silently absent rather than permission-denied. This persisted across a retry, a fresh login, and a full `wsl --shutdown` and restart, ruling out simple caching as the cause. **None of these three issues reflect a problem with the Wazuh configuration itself** - every cert path and filename in `docker-compose.yml` was confirmed correct by reading each container's own real, running config, not guessed. What couldn't be completed is a live deployment on this specific development machine, due to Docker

Desktop/WSL2 environment behavior outside this project's control - documented honestly rather than silently claimed as working. See [logging-dashboard/README.md](#) 's "Live Deployment Attempt" section for the complete troubleshooting record.

Sample log excerpt, word by word

```
<189>Jul 26 2026 09:14:22 MY-KL-HQ-CORE: %SEC-6-IPACCESSLOGP: list GUEST-CONTAINMENT denied tcp 10.10.30.147(52413) -> 10.10.10.15(22), 1 packet
```

Term	Meaning
<189>	The syslog PRI (priority) value - encodes both facility and severity as a single number. 189 decodes to facility 23 (local7, Cisco's default) and severity 5 (notice).
%SEC-6-IPACCESSLOGP	Cisco's structured message format: %FACILITY-SEVERITY-MNEMONIC . This specific mnemonic is what IOS emits when a log-enabled ACL entry matches and denies a packet - the exact real-world message this project's own GUEST-CONTAINMENT ACL would emit if the log keyword were added to its deny lines.
Source/destination with ports	Confirms this specific hit was a GUEST_WIFI device (10.10.30.147) attempting to reach an MGMT-subnet host on port 22 (SSH) - exactly the kind of attempt GUEST-CONTAINMENT exists to block, and exactly the kind of event a SIEM should flag as worth investigating if it repeats.

EXAM ANGLE — CORRELATION IS THE POINT, NOT JUST STORAGE

Any single log line in these sample files looks routine in isolation - one denied packet, one failed login. A SIEM's actual value is catching the *pattern*: the same source IP appearing in the ACL-deny log, the SSH-failure log, and a port-security violation within the same few minutes tells a very different story than any one of those alone. This is precisely why Security+ Domain 4.0 treats log aggregation and correlation as a distinct security control, not merely a convenience over checking each device individually.

EXPECTED RESULT (WERE THIS STACK EVER RUN)

Wazuh's built-in SSH brute-force correlation rule (rule group `authentication_failures`) would fire against the failure burst in `ssh-auth.log` from `TH-BKK-ACC` , generating a dashboard alert - see [logging-dashboard/README.md](#) for the full replay procedure and what else to look for.

9. Design Decisions & Honesty Notes

Decision	Reasoning
Netmiko/Ansible written but never executed live	Confirmed via direct investigation (not assumed) that Packet Tracer 9.0.0's <code>PT-Cloud</code> has no real-NIC bridge option - genuinely correct, real-device-ready code with no live target available, stated plainly rather than pretending otherwise
<code>device_inventory.py</code> genuinely runs	The one Phase 5 deliverable needing no live connection at all - its output is committed as real, verified proof, not a hypothetical
Credentials read from environment variables, never hardcoded	Same principle applied throughout Phase 4 (<code>random_password</code> + <code>Secrets Manager</code> , never a plaintext password in a <code>.tf</code> file) - <code>devices.py</code> and <code>inventory.ini</code> both raise/fail rather than silently defaulting a password
Bulk ACL script accounts for MY-KL-HQ-CORE's different ACL application point	Automation that assumes "identical devices behave identically" is a real-world source of bugs - this script only edits ACL entries (identical logic for all 3 devices), explicitly leaving ACL <i>application</i> (which differs per Phase 3's SVI workaround) as a separate, per-device concern
Wazuh fed sample logs, not a live feed	Same Packet Tracer limitation as Netmiko/Ansible - logs reconstructed in genuine Cisco IOS syslog format from this project's own real design, clearly labeled as samples

10. Appendix — Quick Reference

File	Purpose
<code>python-scripts/devices.py</code>	Shared device inventory + connection-parameter builder, imported by the other scripts
<code>python-scripts/backup_configs.py</code>	Netmiko - config backup across all 6 devices
<code>python-scripts/bulk_acl_update.py</code>	Netmiko - bulk ACL push across 3 devices
<code>python-scripts/device_inventory.py</code>	Parses local <code>.cfg</code> files -> <code>device_inventory.json</code> (genuinely runs)
<code>ansible-playbooks/inventory.ini</code>	Ansible inventory - all 6 devices, grouped by role
<code>ansible-playbooks/deploy_vlan_acl.yml</code>	Declarative VLAN + ACL deployment
<code>logging-dashboard/docker-compose.yml</code>	Single-node Wazuh stack (indexer + manager + dashboard)
<code>logging-dashboard/sample-logs/</code>	Representative Cisco IOS syslog samples, 3 files

AREHI-SECOPS — Phase 5 Full-Depth Automation & Certification Guide · Generated as a certification-study companion to the `04-automation-and-secops/` code and planning documentation in this repository. This closes out all 5 phases of the AREHI-SECOPS project.